Models and Tools for Cyber-Physical Systems

Chapter 3: Safety Requirements

Instructors: Suman Sourav, Jonathan Julian Huerta y Munive
{sumansourav, jjhymuni}@cs.aau.dk

Original slides courtesy of Rajeev Alur
alur@cis.upenn.edu

Summary

- Requirments: collection of predicates about a system whose satisfaction implies that the system is acceptable.
  - Safety: "nothing bad ever happens" ← our focus
  - Liveness: "something good eventually happens"

- Verification problem: given a requirement P and a system/model C, prove/disprove that C satisfies P

- Definition of transition system vs extended-state machine

- Definition of reachable states of the system C ($Reach(C)$), reachable property P ($Reach(C) \cap P \neq \emptyset$), and invariant I ($Reach(C) \subseteq I$)

- Definition of inductive invariant and strengthening strategy to obtain inductive invariants

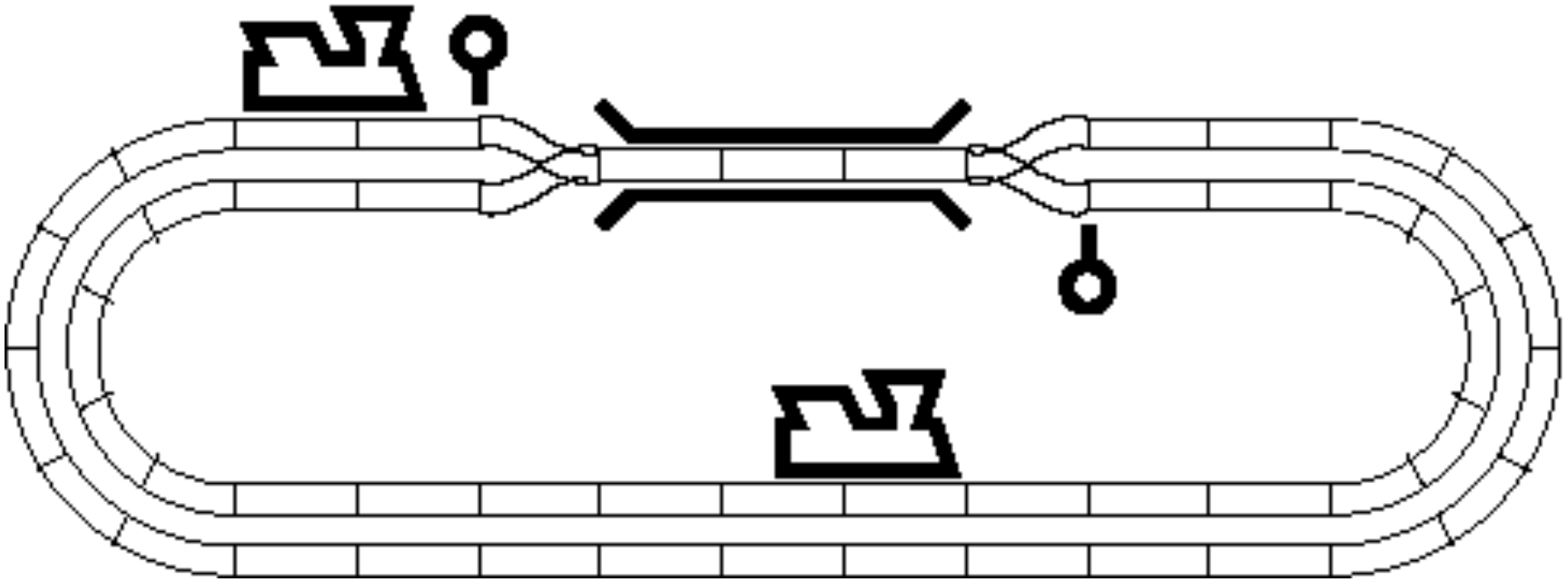
Case Study: Railroad Controller

Requirements-Based Design

- Systematic approach to design of systems
- Given:
  - Input/output interface of system C to be designed
  - Model E of the environment
  - Safety property φ of the composite system

- Design problem: Fill in details of C (state variables, initialization, and update) so that C || E satisfies the invariant φ

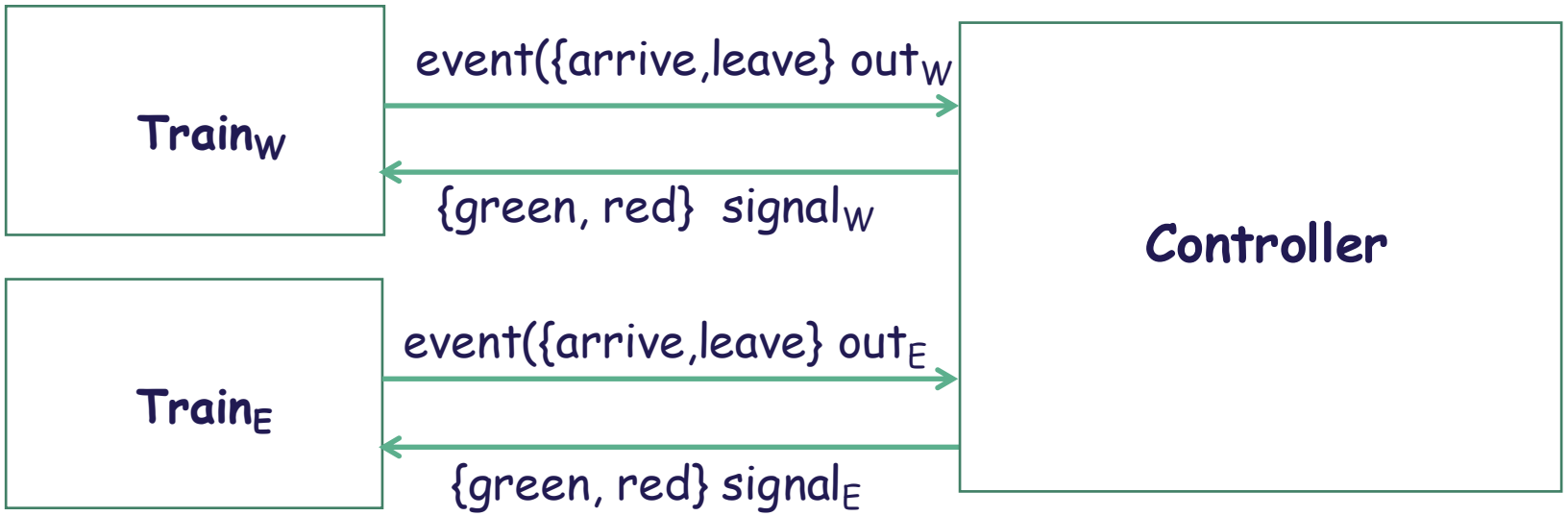
Railroad Controller Example



Train Model

- From the perspective of the controller, the train is initially (far) away
- The train can be away for an arbitrarily long period
- When the train gets close, it communicates with the controller via an event arrive, and now it is in a different state wait
- In the wait state, the train monitors the signal:
  - If the signal is green, the train enters the bridge (state bridge)
  - If the signal is red, the train continues to wait
- A train can stay on the bridge for a duration that is not exactly known (and not influenced by the controller)
- When the train leaves the bridge, it communicates with the controller via an event leave, and goes back to the away state
- This behavior repeats: a train that is away may again request entry
- Both trains have symmetric behavior

Controller Design Problem

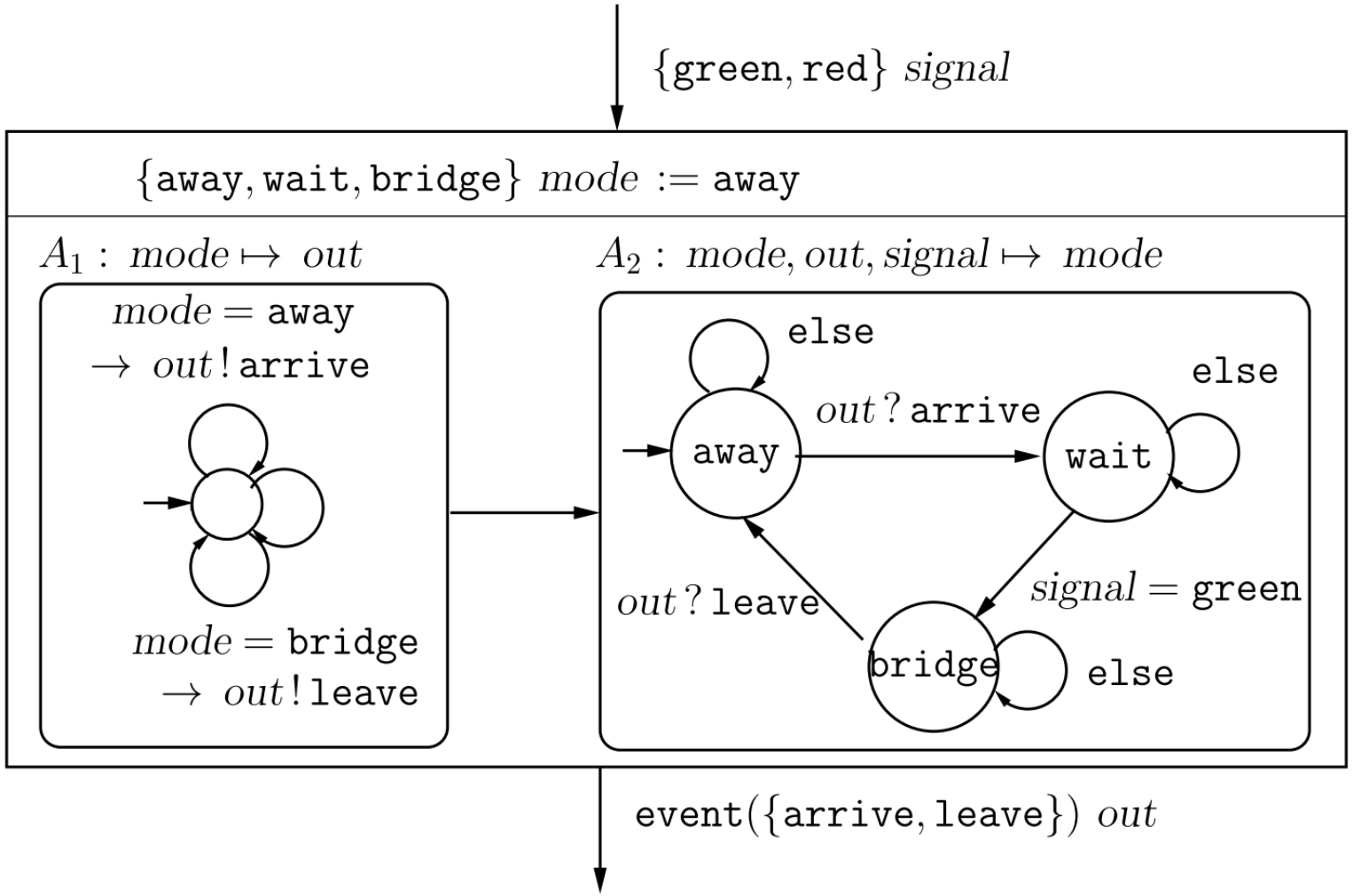


Safety requirement: Trains are not on the bridge simultaneously

$$\sim(\text{mode}_W = \text{bridge} \ \& \ \text{mode}_E = \text{bridge})$$

This is an invariant requirement

Synchronous Component "Train"

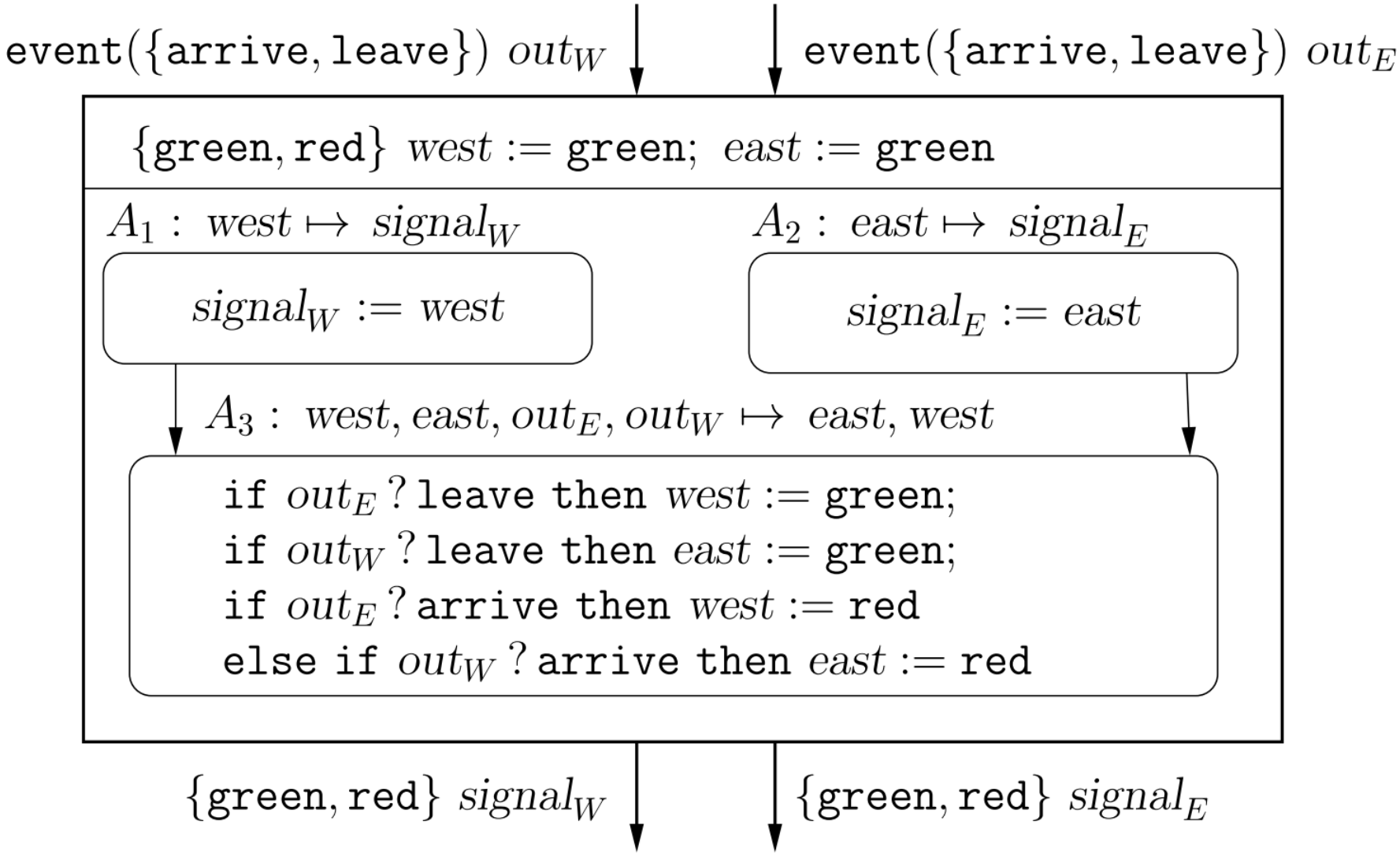


First Attempt at Controller Design

- Controller maintains two state variables to track the state of each signal
- Both state variables are initially green
- Set the output variables based on the corresponding state variables
- If a train arrives, then update the opposite signal variable to red to block the other train from entering
- If a train leaves, reset the opposite signal variable to green

- What happens if both trains arrive simultaneously?
  - Tie breaker: Give priority to east train

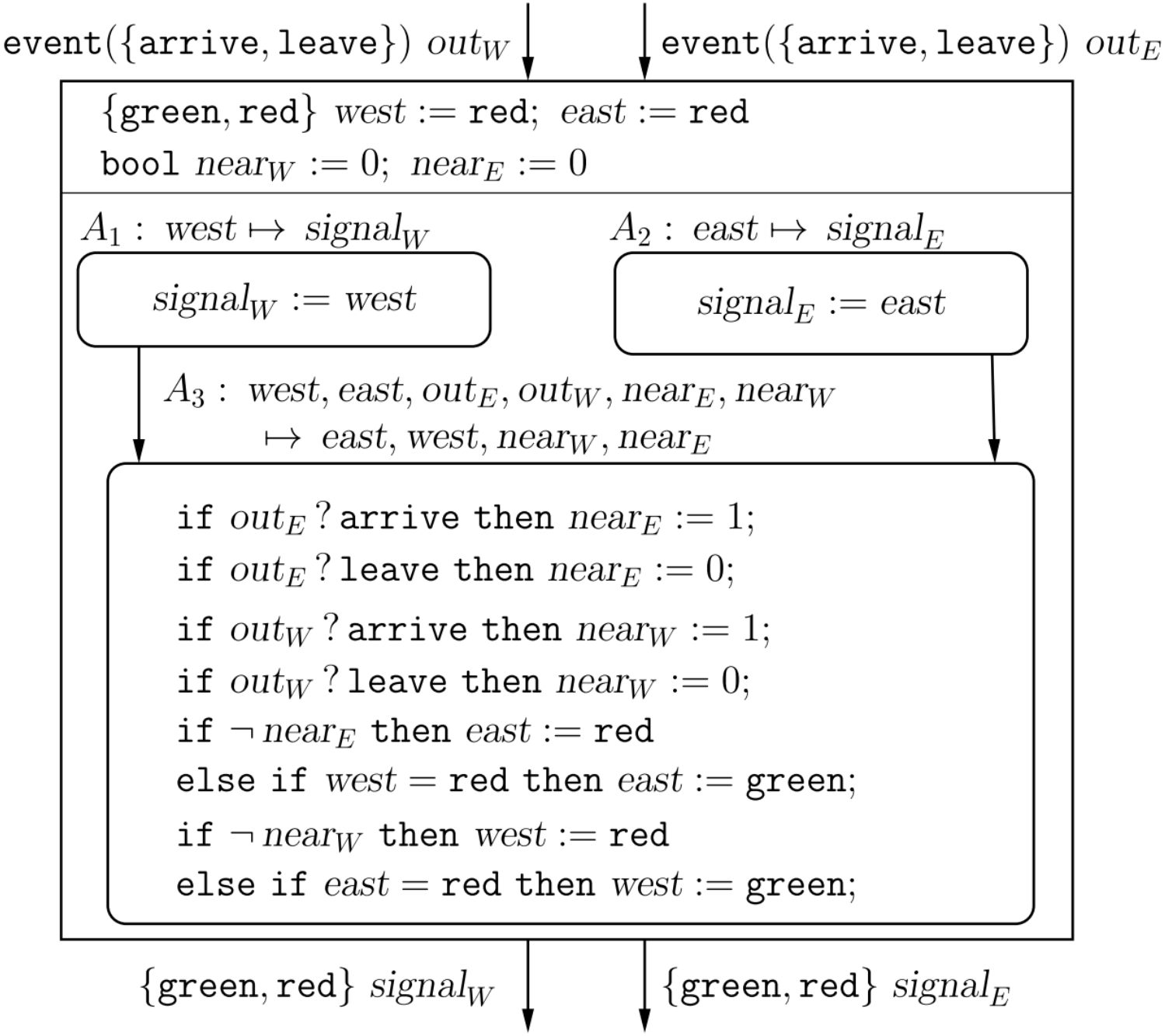
Synchronous Component "Controller1"



west	east	$mode_W$	$mode_E$
green	green	away	away
↓ green	↓ green	↓ arrive!	↓ arrive!
red	green	wait	wait
↓ red	↓ green	↓	↓
red	green	wait	bridge
↓ red	↓ green	↓	↓ leave!
green	green	wait	away
↓ green	↓ green	↓	↓ arrive!
red	green	bridge	wait
↓ red	↓ green	↓	↓
red	green	bridge	bridge

Second Attempt at Controller Design

- What went wrong the first time? Controller did not remember whether a train was waiting at each entrance
- We fix this by adding a new Boolean variable $near_W$ that remembers whether the west train wants to use the bridge
  - Is initially 0
  - Is changed to 1 when the west train issues arrive
  - Is reset back to 0 when the west train issues leave
- Invariant: $mode_W$ = away if and only if $near_W = 0$
- We add another,symmetric variable $near_E$
- We also now keep both signals red by default
- A signal is changed to green if the corresponding train is near and the other signal is red, and changed back to red when the train is away
- Still resolve simultaneous arrivals by preferring the east train


event({arrive, leave}) out_W
event({arrive, leave}) out_E
{green, red} $west$:= red; $east$:= red
bool $near_W$:= 0; $near_E$:= 0
A_1 : $west \mapsto signal_W$
$signal_W$:= $west$
A_2 : $east \mapsto signal_E$
$signal_E$:= $east$
A_3 : $west, east, out_E, out_W, near_E, near_W$
$\mapsto east, west, near_W, near_E$
if out_E ? arrive then $near_E$:= 1;
if out_E ? leave then $near_E$:= 0;
if out_W ? arrive then $near_W$:= 1;
if out_W ? leave then $near_W$:= 0;
if $\neg near_E$ then $east$:= red
else if $west$ = red then $east$:= green;
if $\neg near_W$ then $west$:= red
else if $east$ = red then $west$:= green;
{green, red} $signal_W$
{green, red} $signal_E$

Properties of Controller2

- The system RailRoadSystem2 = Controller2 || TrainW || TrainE satisfies the safety requirement (invariant)
- Additional (alternative) requirements we could be interested in
  1. If the west train is waiting, the west signal will eventually become green
  2. If the west train is waiting for its signal to turn green, the other train should not be allowed on the bridge more than once
- Requirement 1 is a liveness property (see Chapter 4)
- Requirement 2 is a safety property: its violation can be demonstrated by a (finite) execution in which the east train enters, leaves, and enters again while the west train keeps waiting with its signal red
  - Not an invariant (not expressible over state variables only)

Safety Monitors

Safety Monitors

- Before we modelled "The trains are not on the bridge simultaneously" as

$$\neg\,(mode_E = bridge \wedge mode_W = bridge)$$

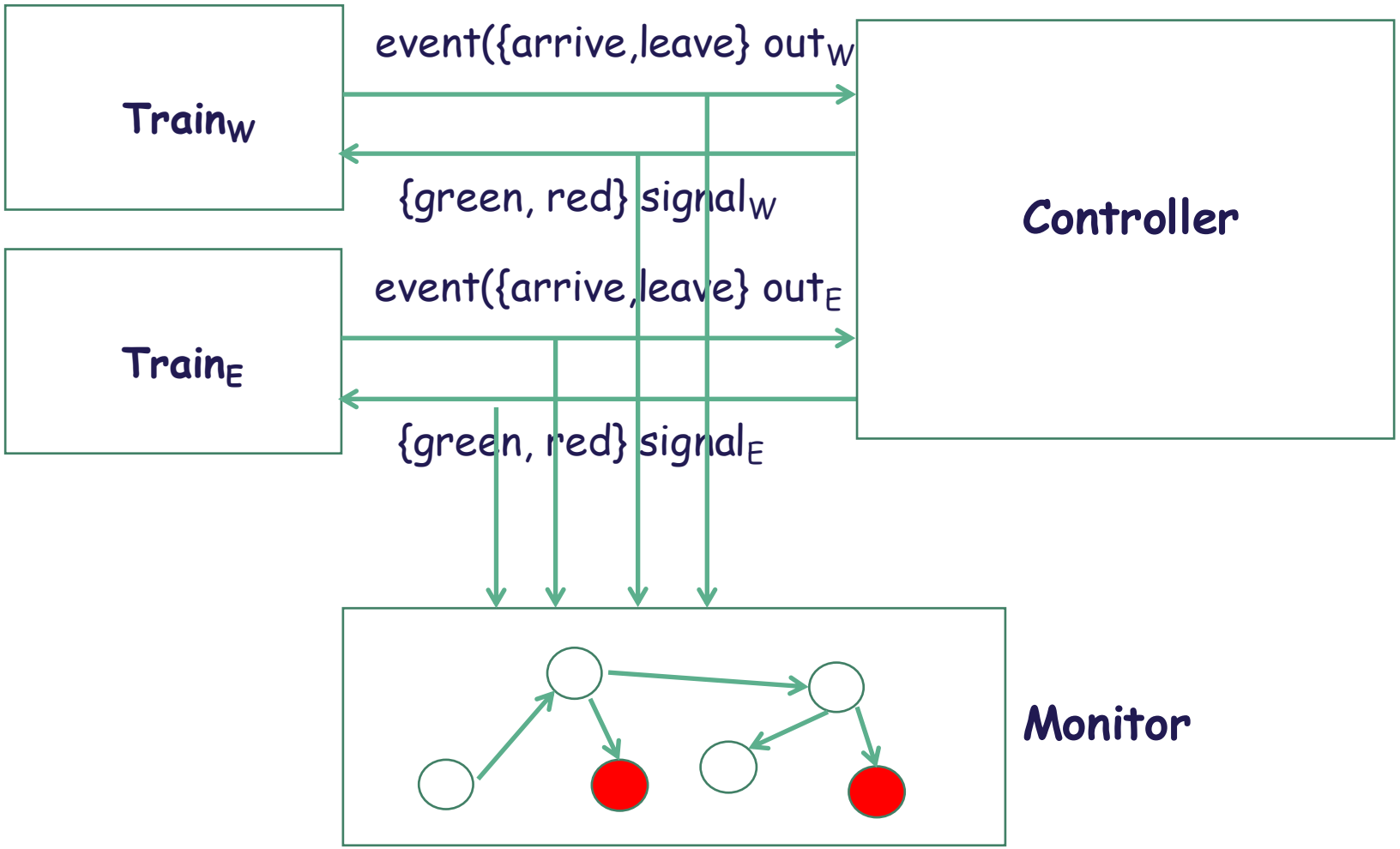
- However, the property "While a train is waiting with its signal red, the other train should not leave the bridge twice" cannot be formalised directly:

$$mode_T \in \{away, wait, bridge\} \approxeq \{a, w, b\}$$
$$out_T \in \{arrive, leave\}$$
$$signal \in \{red, green\} \approxeq \{r, g\}$$
$$near_T \in \{1, 0\}$$

$$mode_E = wait \wedge signal_E = red$$
$$\wedge\, mode_W = leave \wedge ?$$

- Solution: add a system M so that we can specify a property about M || C

Safety Monitor

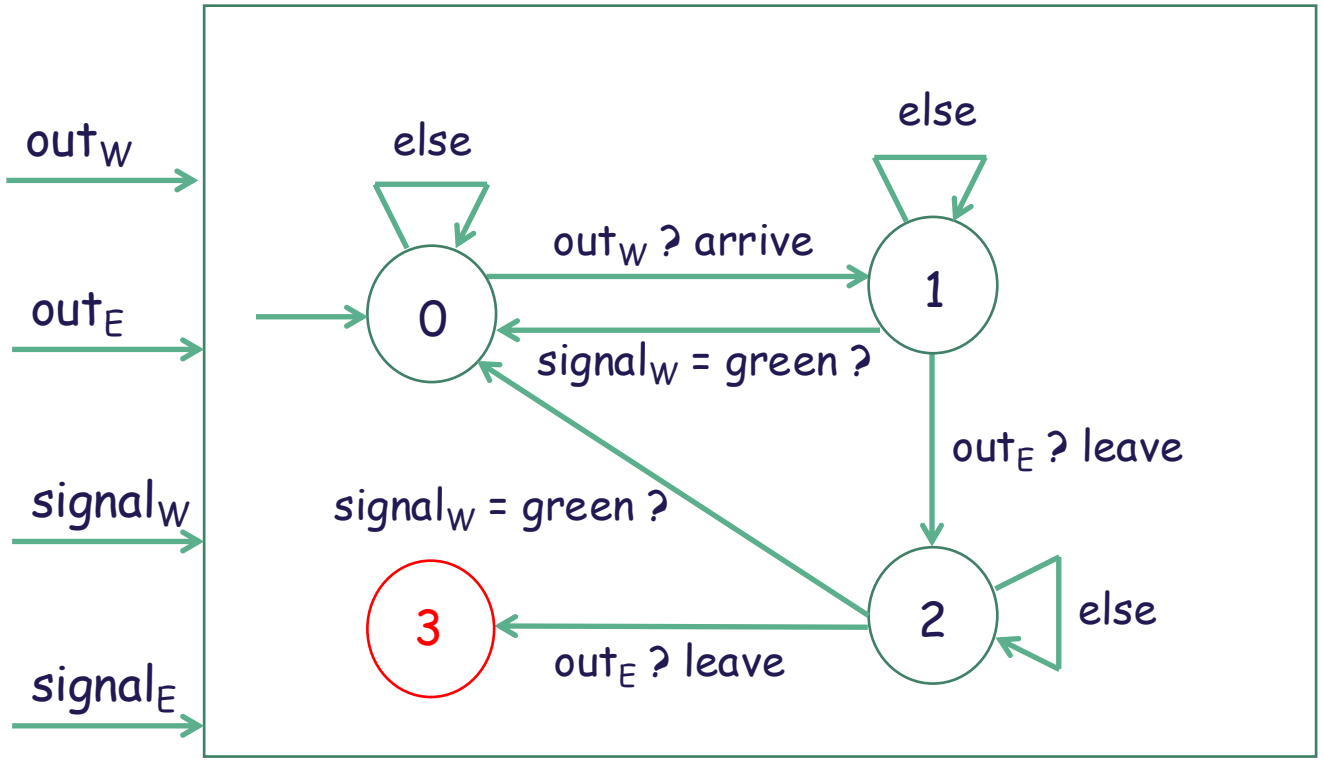


Safety Monitor

In the context of the book and Rajeev Alur's reactive components:

- A monitor observes the inputs/outputs of a system and enters an error state if undesirable behavior is detected
- A monitor M is typically specified as an extended state machine
  1. The input variables of M are the input/output variables of the system being monitored
  2. The output variables of M are disjoint from those of the system (the monitor should not influence the system)
  3. A subset F of the modes of the state machine is accepting ("fail")
- Undesirable behavior: An execution that leads to a monitor state in F
- Safety verification: Check whether "monitor.mode not in F" is an invariant of system C || M

Monitor to Check Train Requirement 2



"If the west train is waiting for its signal to turn green, the east train should not be allowed on the bridge more than once"

Error execution: As west train waits, east train is allowed on bridge twice

Safety Monitors (final remark)

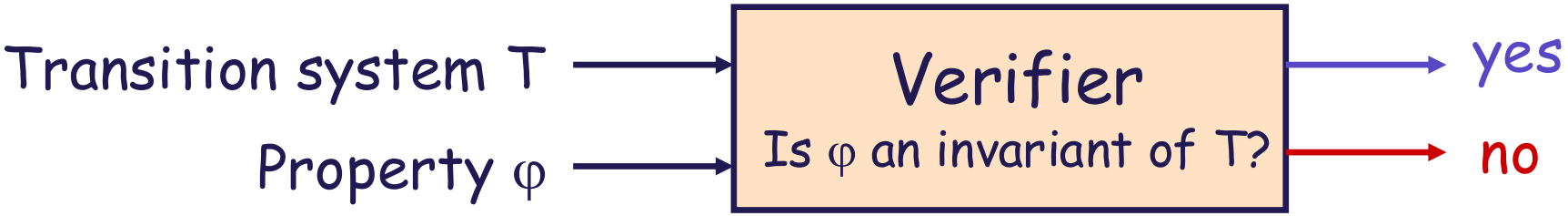
"If both trains are waiting, then the controller must allow some train to eventually enter the bridge."

Must be bounded!
Otherwise, liveness (chapter 5)

"If both trains are waiting and both signals are red, then the controller must turn one of the signals to green in the next round"

Complexity of Verification Problems

Invariant Verification Problem



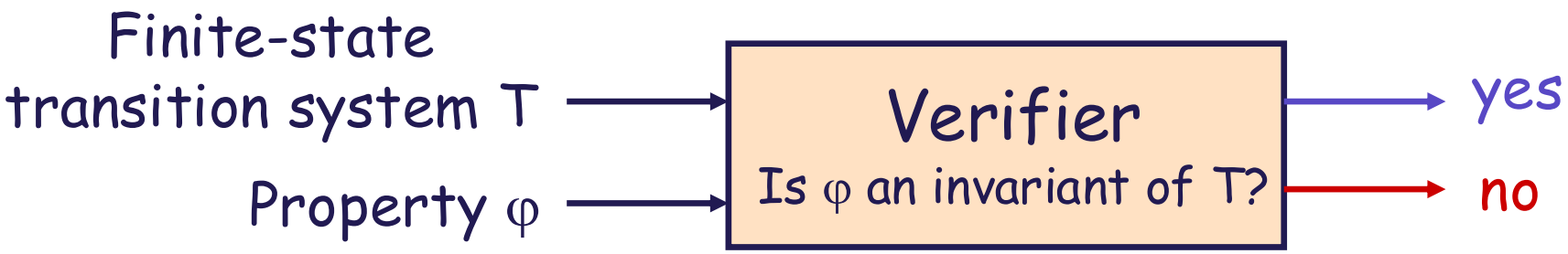
Can such a verifier exist?

If so, what is the computational complexity of the verification problem?

Theorem: Invariant verification problem is undecidable

- If some state variables in T are of type int, then T can correspond to an arbitrary program (or Turing machine)
- Intuition: there is no a priori bound on number of reachable states in such case, so examining all reachable states is not possible

Finite-State Invariant Verification Problem



Theorem: Invariant verification problem for finite-state systems is decidable

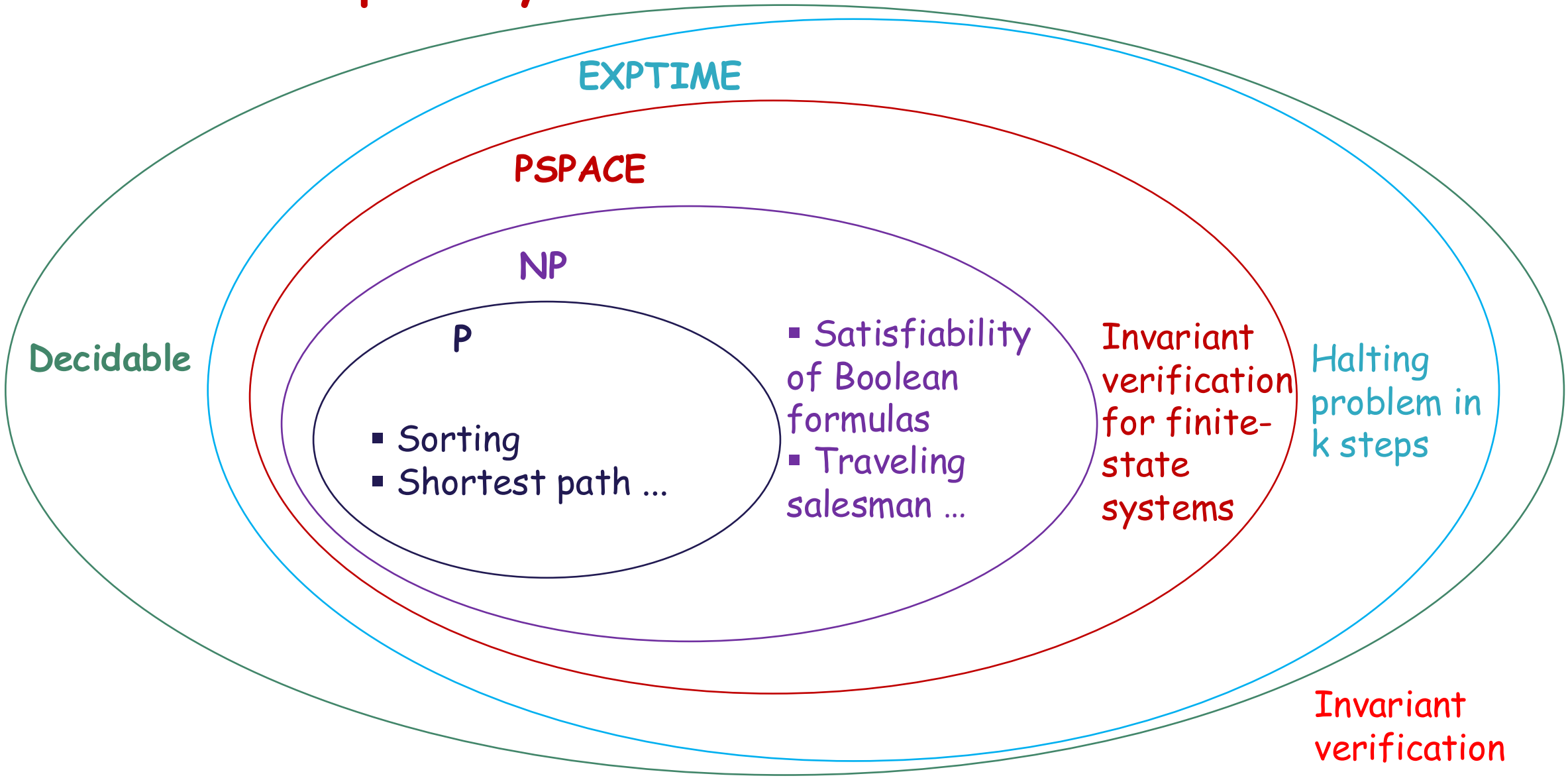
If T has k Boolean variables, then total number of states is 2^k

Verifier can systematically search through all possible states

Complexity is exponential (more precisely PSPACE, a class of problems that are presumably harder than NP problems)

- knowing whether I is an invariant for T requires space of up to polynomial size of the input description to be solved
- This is true regardless of modeling language, or whether systems are deterministic or nondeterministic

Complexity Classes



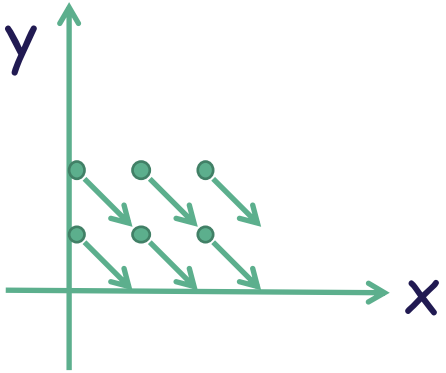
Symbolic Invariant Verification

Solving Invariant Verification

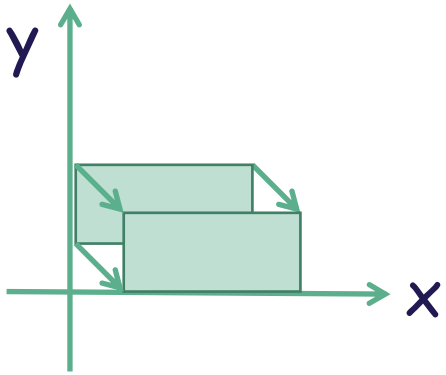
- Establishing that the system is safe is important, but there is no efficient algorithm to solve the verification problem
- Solution 1: Use simulation-based analysis (testing and monitoring)
  - Simulate the model multiple times and check that each state encountered on each execution satisfies the desired invariant
  - Practical in most real-world applications, but no guarantee and known to miss hard-to-find bugs
- Solution 2: Write a formal proof using inductive invariants
  - Only partial tool support, so requires considerable effort
  - Some successes: verified operating systems (e.g., seL4 microkernel) and compilers (e.g., CompCert C compiler)
- Solution 3: Exhaustive search through state space
  - Fully automated, but has scalability limitations
  - Complementary to simulation, increasingly used in industry
  - On-the-fly enumerative search, Symbolic search

Computing Reachable States

- Search algorithm can start with initial states, and explore transitions out of initial states, in a systematic manner
- Example: integer state variables x, y; initially 0 <= x <= 2 and 1 <= y <= 2, and each transition increments x and decrements y



Enumerative:
Consider individual states



Symbolic:
Consider set of states

Reachable subgraph

3 x 3 x 2 x 2 x 2 x 2 = 144 possible states

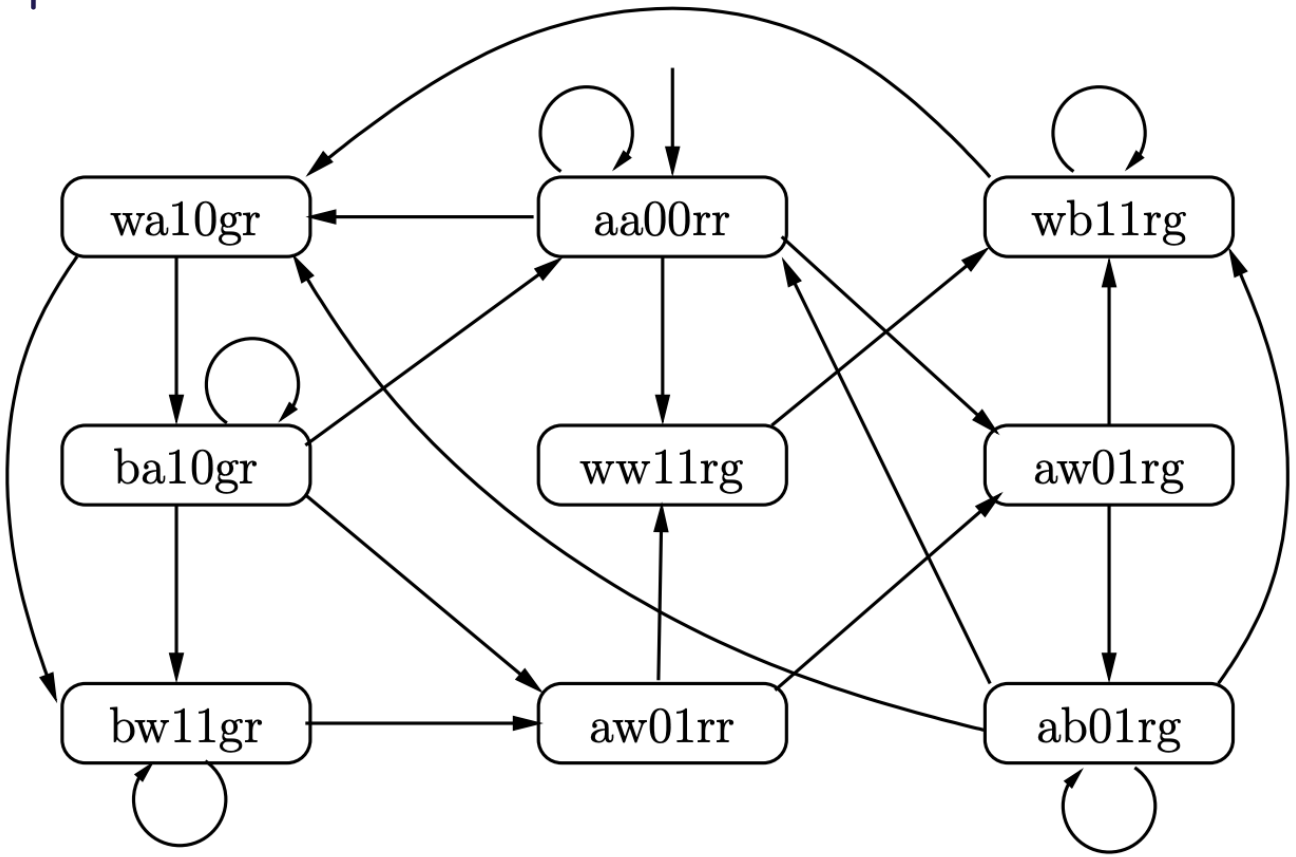
$(mode_W, mode_E, near_W, near_E, west, east)$

$mode_T \in \{away, wait, bridge\} \approxeq \{a, w, b\}$

$out_T \in \{arrive, leave\}$

$signal \in \{red, green\} \approxeq \{r, g\}$

$near_T \in \{1, 0\}$

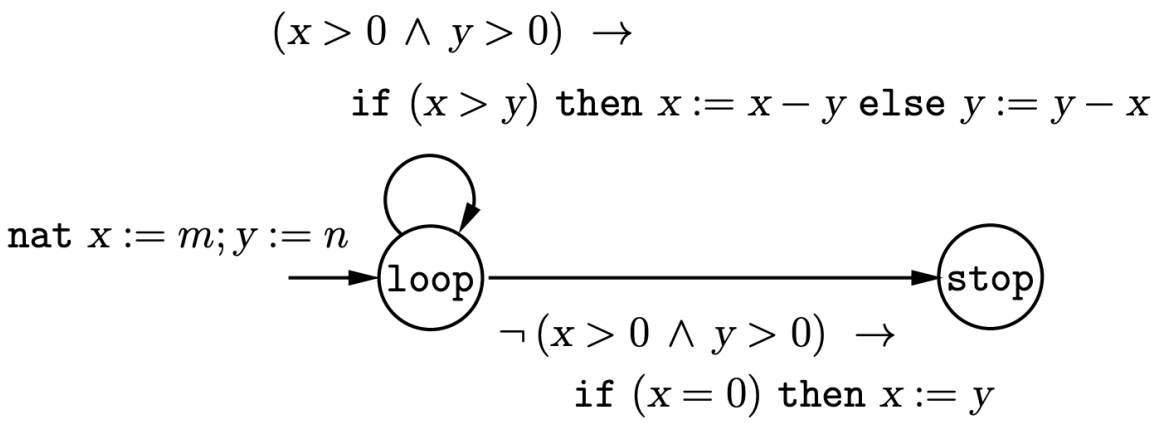


- This time the reachable states were few, but the space can be considerably large.
- We can reduce some cases symbolically, e.g. represent $x \in \{20,21,22,\ldots,99\}$ as $x \in [20,99]$

Toward a Symbolic Search Algorithm

- We need a way to represent and manipulate sets of states
- Basic data structure: region
  - For now, a region is given by a formula (= Boolean-valued expression)
  - Example 1: (x & y) | z, where x, y, z are Boolean variables
  - Example 2: x <= y+1 & 0 <= x <= 3, where x, y are int/real variables
- Which operations are needed on regions?
  - Will become clear as we develop the algorithm
- "Better" implementation of regions other than formulas?
  - Binary decision diagrams (BDDs) for Boolean variables
  - Intervals or polyhedra (matrices) for real-valued variables
  - More complex designs depending on the system dynamics

Symbolic Representation of Transition System



How to express the region (formula) of a program/transition?

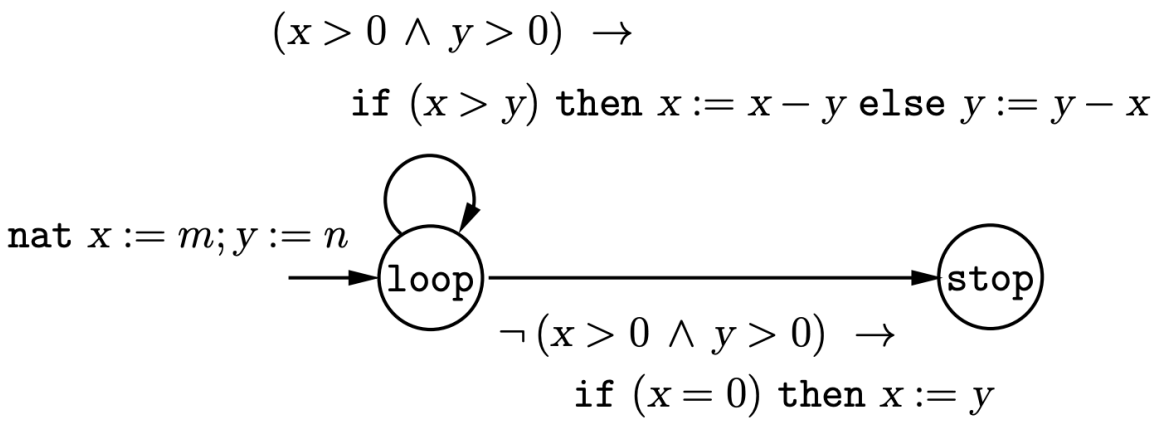
```
def GCD(m, n)
    x:=m; y:=n;
    while (x > 0 ∧ y > 0):
        if x > y then x := x-y else y:=y-x
    if (x = 0) then x := y
    return x
```

Initialization formula:

$\varphi_I = (x = m \wedge y = n \wedge mode = loop)$

Do not forget the mode!

Symbolic Representation of Transition System



How to express the region (formula) of a program/transition?

Transition formula:

$\varphi_T = ?$

```
def GCD(m, n)
    x:=m; y:=n;
    while (x > 0 ∧ y > 0):
        if x > y then x := x-y else y:=y-x
    if (x = 0) then x := y
    return x
```

if x > y then x := x-y else y:=y-x

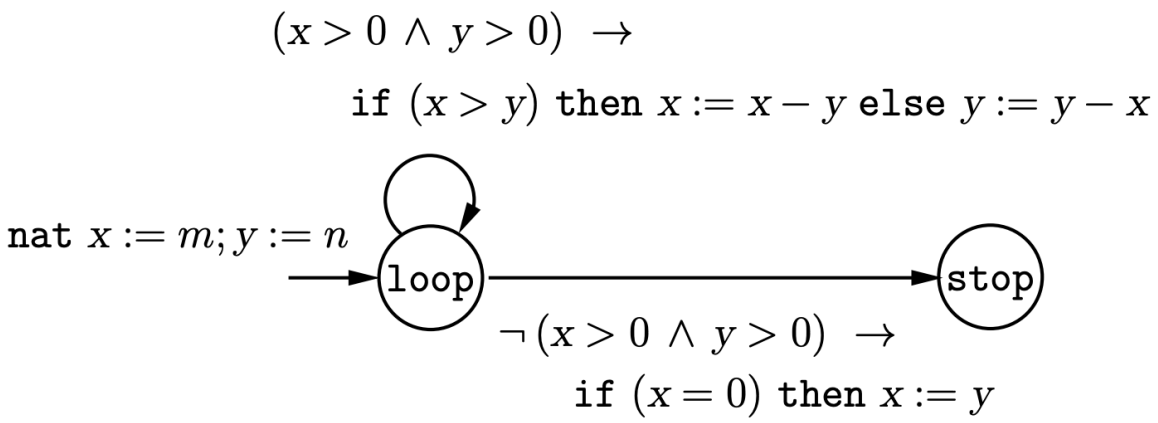
$\varphi_{then1} = \quad x > y \wedge x' = x - y \wedge y' = y$

$\varphi_{else1} = \neg(x > y) \wedge y' = y - x \wedge x' = x$

$\varphi_{if1} = \varphi_{then1} \vee \varphi_{else1}$

Do not forget to indicate the remaining variables!

Symbolic Representation of Transition System



How to express the region (formula) of a program/transition?

Transition formula:

$$\varphi_T = \varphi_{loop} \vee \varphi_{stop}$$

$$\varphi_{if1} = \varphi_{then} \vee \varphi_{else}$$

```
def GCD(m, n)
    x:=m; y:=n;
    while (x > 0 ∧ y > 0):
        if x > y then x := x-y else y:=y-x
    if (x = 0) then x := y
    return x
```

$$\varphi_{loop} = x > 0 \wedge y > 0 \wedge mode = loop \wedge \varphi_{if1} \wedge mode' = loop$$

$$\varphi_{stop} = \neg(x > 0 \wedge y > 0) \wedge mode = loop \wedge \varphi_{if2} \wedge mode' = stop$$

Symbolic Representation of Transition System

- Consider a transition system T with variables S
- How to represent the set of initial states?
  - Formula φ_I over variables S
  - Easy to obtain from the initialization description Init
  - Recall other example "if (x < m) then {x := x+1; y := y-1}"
  - And the initialization formula φ_I is x = 0 & y = m
- How to represent the set of transitions?
  - Convention: Primed variable denotes updated value (i.e., describes the target state of a transition)
  - Transitions are given by a formula φ_T over variables $S \cup S'$
  - Example: In one transition, increment x and decrement y:
    - x' = x+1 & y' = y-1
    - A pair of states (s, t) over x, y satisfies this formula exactly when t(x) = s(x)+1 and t(y) = s(y)-1
- If T has k variables, then a set of states is given by a region over k variables, and transitions are given by a region over 2k variables

Transition Formula

- Suppose a single step is given by:

  if (x > 0 & y > 0) then { if (x > y) then x := x-y else y := y-x }
- Find a formula φ_T over variables x, y, x′, y′ to capture this
- Assignment x := x-y corresponds to constraint x′ = x-y
- But this is not enough. If y is not assigned explicitly, it stays unchanged. In formulas, we must say y′ = y to enforce this
- Assignment x := x-y corresponds to formula x′ = x-y & y′ = y
- Assignment y := y-x corresponds to formula x′ = x & y′ = y-x
- Conditional statement if (x > y) then x := x-y else y := y-x becomes

  ψ: [x > y & x′ = x-y & y′ = y] | [~(x > y) & x′ = x & y′ = y-x]
- Desired transition formula φ_T for the entire statement is

  [x > 0 & y > 0 & ψ] | [~(x > 0 & y > 0) & x′ = x & y′ = y]

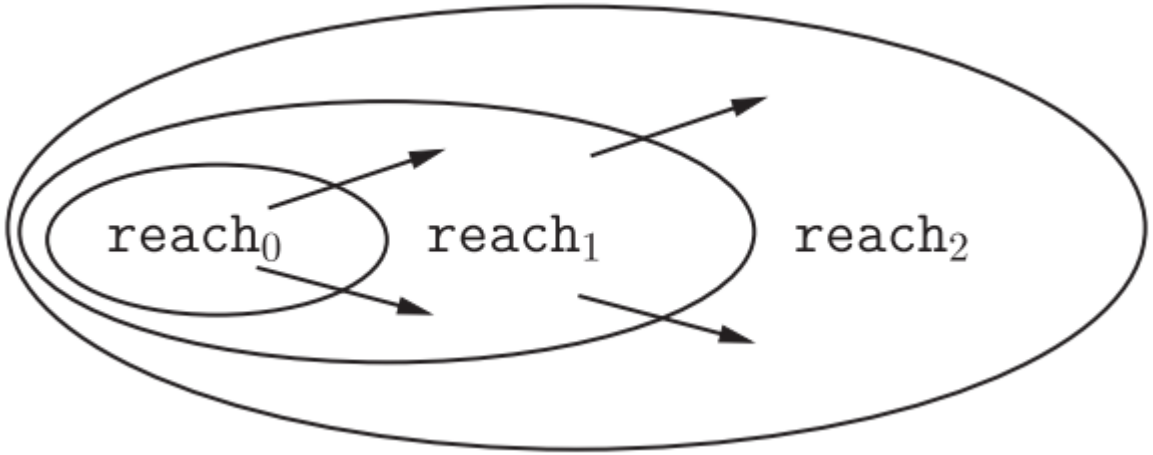
Symbolic Transition System

- Region over variables X is a data structure that represents a set of states assigning values to X
- Transition system T = (S, Init, Trans) can be represented by
  - Region φ_I over S for initial states
  - Region φ_T over $S \cup S'$ for transitions
- Symbolic representation can be compiled automatically from code
  - To get φ_T from reaction description of a Synchronous Reactive Component, local/input/output variables must be existentially quantified (see textbook for examples)

Image Computation

- Given a region A, define Post(A, φ_T) as the set of successors of states in A

  Post(A, φ_T) = { t | there exists a state s in A and a transition (s, t) }
- Core problem in symbolic search: Given A and the transition formula, how to compute Post(A, φ_T)?
- If we can implement this operation, the set of all reachable states can be enumerated iteratively (e.g., by breadth-first search)



reach$_0$ = initial states

each reach$_{i+1}$ obtained from reach$_i$ by applying Post

Image Computation: Example 1

- Suppose T has a single variable x of type real
- Consider transition formula φ_T : x' = 2x+1
- How to compute Post(A, φ_T) systematically, where A is given by 0 <= x <= 10
- Step 1: Take conjunction of (i.e., intersect) A with φ_T
  - Result: 0 <= x <= 10 & x' = 2x+1
  - Intuition: this represents all transitions (x, x') starting in region A
- Step 2: Existentially quantify x (= projection) and simplify
  - Result: ∃x: 0 <= x <= 10 & x' = 2x+1
  - Intuition: formula only considers those x' for which there exists some x such that x is in A and (x, x') is a transition
  - Result simplifies to 0 <= (x'-1)/2 <= 10, which is equivalent to 1 <= x' <= 21
- Step 3: Rename x' to x (because we want Post(A) to be a formula involving x)
  - Result: 1 <= x <= 21
- We can check that [1, 21] is exactly the image of [0, 10] if x goes to 2x+1

Image Computation: Example 2

- Suppose T has variables x and y of type real
- Consider transition formula φ_T : $x' = x+1 \;\&\; y' = x$
- Suppose A is given by $0 <= x <= 4 \;\&\; y <= 7$

- Step 1: Take conjunction of (i.e., intersect) A with φ_T
  - Result: $0 <= x <= 4 \;\&\; y <= 7 \;\&\; x' = x+1 \;\&\; y' = x$

- Step 2: Existentially quantify x and y (= projection) and simplify
  - Let us first project out x: $\exists x: 0 <= x <= 4 \;\&\; y <= 7 \;\&\; x' = x+1 \;\&\; y' = x$
    This simplifies to $0 <= y' <= 4 \;\&\; y <= 7 \;\&\; x' = y'+1$
  - Now project out y: $\exists y: 0 <= y' <= 4 \;\&\; y <= 7 \;\&\; x' = y'+1$
    This simplifies to $0 <= y' <= 4 \;\&\; x' = y'+1$

- Step 3: Rename x' to x and y' to y
  - Result: $0 <= y <= 4 \;\&\; x = y+1$

Operations on Regions

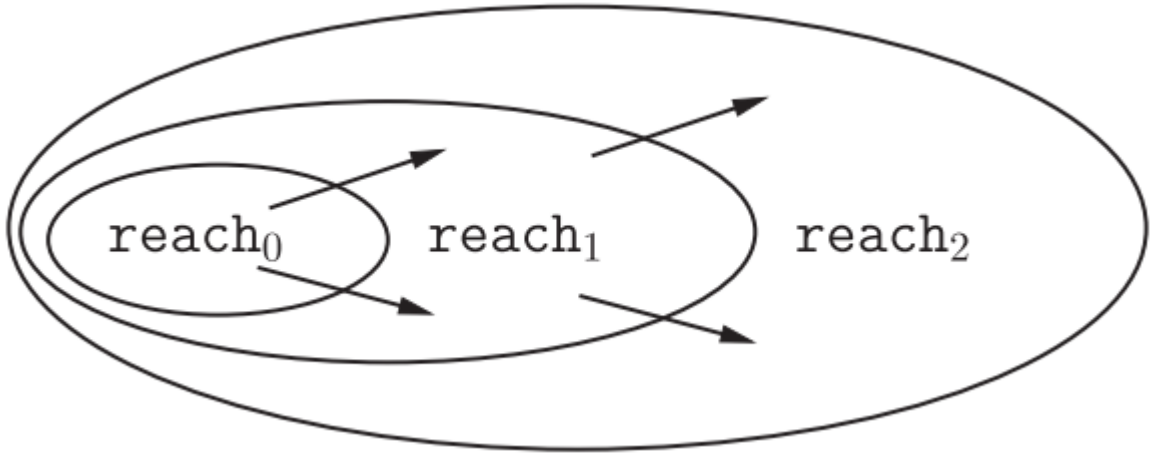
- In general, we want to represent sets of states by a data type reg, which should support the following operations
- Disj(A,B): Returns region containing states in A or in B
  - For formulas, this is just "A | B"
- Conj(A,B): Returns region containing states that are in both A and B
  - For formulas, this is just "A & B"
- Diff(A,B): Returns region containing states in A but not in B
  - For formulas, this is "A & ~B"
- IsEmpty(A): Returns 0 if region A contains some state, and 1 otherwise
  - For formulas, this requires testing "satisfiability": can the variables be assigned values to make the formula true?
- Exists(A,X): Returns projection of A by quantifying variables in X
  - For formulas, this requires "quantifier elimination"
- Rename(A,X,Y): Rename variables in X to corresponding variables in Y
  - For formulas, this is textual substitution

Symbolic Image Computation

- Given:
  - Region A of type reg over state variables V_S
  - Region φ_T of type reg over $V_S \cup V_S'$

- Post(A, φ_T) = Rename(Exists(Conj(A, φ_T), V_S), V_S', V_S)
  1. Take conjunction of A and φ_T
  2. Project out the variables in V_S using existential quantification
  3. Rename primed variables to get a region over V_S

Symbolic Breadth-First-Search Algorithm



reach$_0$ = initial states

each reach$_{i+1}$ obtained from reach$_i$ by applying Post

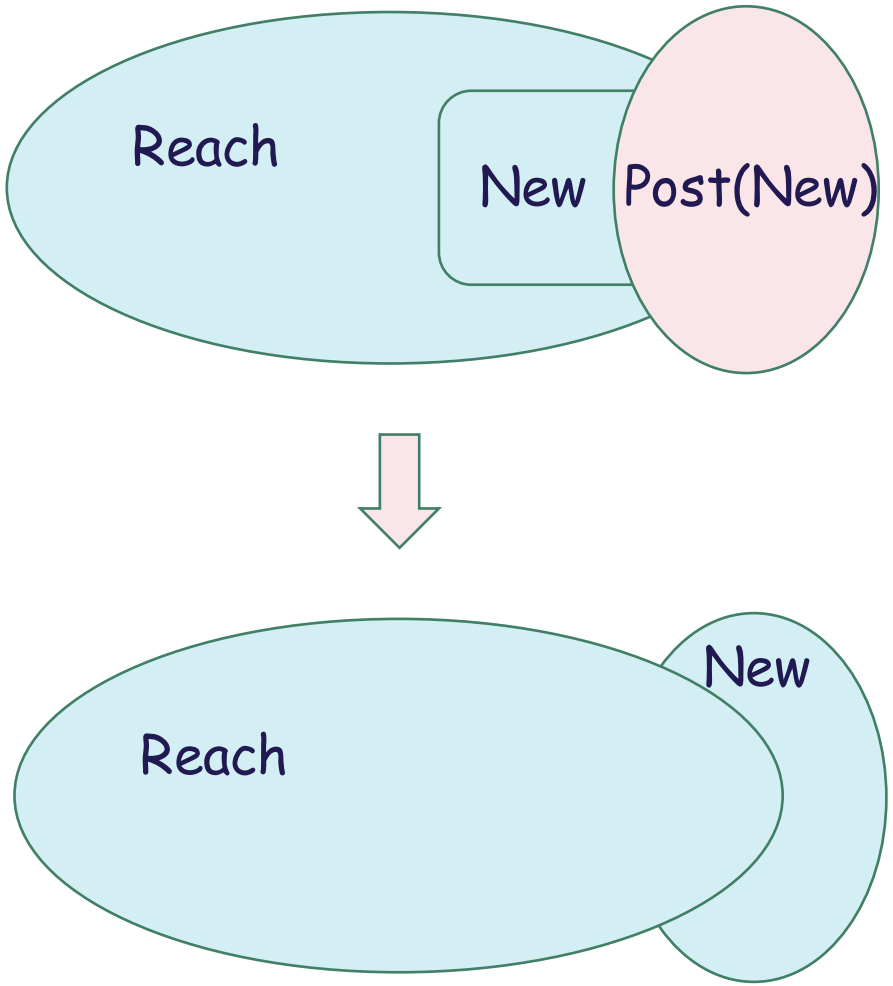
- Algorithm for checking whether a property φ is an invariant of T
- Same as checking if the "error" states $\sim\varphi$ are reachable
- We need to check at every step if error states are reached; if so, stop
- If no new states are encountered, then also stop (invariant satisfied)
- May run forever

Symbolic BFS Algorithm

Given region φ_I over V_S, region φ_T over $V_S \cup V_S'$, and region φ over V_S,
if φ is reachable in T then return 1, else return 0

```
reg Reach := φI;  /* states found to be reachable */
reg New := φI;  /* states not yet explored */
while IsEmpty(New) = 0 {  /* while there are states to be explored */
   if IsEmpty(Conj(New, φ)) = 0  /* property φ is reachable */
      then return 1;
   New := Diff(Post(New, φT), Reach);
     /* These are states in the post-image of New, but not
         previously found reachable, so they have to be explored */
   Reach := Disj(Reach, New);  /* add newly found states to Reach */
};
return 0;  /* all states explored without encountering φ */
```

Frontier Computation in Symbolic BFS



Symbolic Search

- Correctness: When the algorithm stops, its answer (whether the property φ is reachable or not) is correct

- Termination: Number of iterations depends on
  - length of shortest execution leading to a state violating φ
  - Diameter: smallest d such that all states reachable within d steps (this may not be bounded if the system is not finite-state)
  - In practice: terminates if one of these numbers is small

- Used in practice for hardware verification and protocol verification

Implementation of Regions

- Key to efficient implementation: How to represent regions?
  - Operations: Disj, Conj, Diff, IsEmpty, Exists, Rename

- Suppose all variables are Booleans

- We can represent regions with formulas (with &, |, ~)
  - Disj, Conj, Diff, Rename easy
  - Exists(φ, x) same as φ [x->0] | φ [x->1]
  - IsEmpty(φ) requires test for satisfiability (SAT)

- SAT is computationally demanding (NP-complete), but more importantly, size of formula representing Reach keeps growing as we apply operations
  - Key to performance: Simplify formulas as much as possible
  - Solution: (reduced ordered) binary decision diagrams ((RO)BDDs)

Summary

- If a property cannot be specified as an invariant for an extended state machine C, we can construct a monitor M for which "monitor.mode not in F" is an invariant of system C || M
- Automating the invariant verification problem is generally undecidable. Even in the finite case, it is a PSPACE-complete problem.
- Proposed solutions include simulation (incomplete), proof using inductive invariants (difficult), exhaustive (symbolic) search
- We discussed a state-based search and moved to a symbolic breadth-first-search algorithm over regions as formulas
- There are optimisations not discussed in this course such as using Reduced Ordered Binary Decision Diagrams as regions